

人工智能本科生实验报告

学号	XXXXXXXX	姓名	XXX
----	----------	----	-----

一、实验题目

编写一个Python程序，实现一阶逻辑的归结算法。该算法需要支持最一般合一（MGU）过程，并能够自动处理给定的逻辑推理问题（如Alpine Club问题等），根据输入的子句集，输出完整的推理证明路径。

二、实验内容

1. 算法原理

归结原理的基本思想是反证法（Proof by Contradiction）。为了证明一个逻辑命题 α 是知识库 KB 的逻辑蕴含（即 $KB \models \alpha$ ），我们只需证明 $(KB \wedge \neg \alpha)$ 是不可满足的（即包含矛盾）。

（1）子句集转换（Clausal Form）：

一阶逻辑公式首先需要转换为子句集。子句是文字（Literal，即原子公式或其否定）的析取（OR）。在一阶逻辑归结中，由于存在变量，不同子句中的变量默认是不相关的，这为后续的替换提供了基础。

（2）最一般合一算法（MGU）：

在命题逻辑中，归结要求两个子句中存在互补对（如 P 和 $\neg P$ ）。但在一阶逻辑中，我们需要通过合一（Unification）寻找一组变量替换 σ ，使得两个看似不同的文字 $P(x)$ 和 $P(a)$ 能够匹配。MGU 是使两个文字一致的最简单、限制最少的替换的集合。

（3）归结步：

给定两个子句 Clause1 和 Clause2，如果不存在变量且有原子 $item \in Clause1$ 且 $\neg item \in Clause2$ ，则二者的归结式为 $(Clause1 - item) \cup (Clause2 - \neg item)$ 。如果存在变量 $var1 \in Clause1$ 或 $var2 \in Clause2$ ，若存在 $\sigma \in MGU()$ 使得变量能够合一，且合一后存在相反的原子公式，则有相似的归结式。通过不断产生

新的归结式，若最终推导出空子句（Empty Clause），则证明成功；反之若一直归结不到新子句且没有得到空子句，则 $KB \models \alpha$ 不成立。

2. 关键代码展示

本实验代码采用了模块化设计，核心功能包括合一、归结循环和回溯优化。

（1）最一般合一：

```
# 判断项的类型
# 0:变量 1:常量 2:函数/谓词
def classify_item(s:str):
    if s in variables:
        return 0
    if '(' not in s:
        return 1
    return 2
```

```
# 获取替换规则
def unify(term1, term2, subst = None):
    if subst is None:
        subst = {}
    # 如果已存在替换，尝试递归寻找
    if term1 in subst:
        return unify(subst[term1], term2, subst)
    if term2 in subst:
        return unify(term1, subst[term2], subst)
    # 两项均为变量，直接替换
    if classify_item(term1) == 0 and classify_item(term2) == 0:
        subst[term1] = term2
        return True, subst
    # 变量+常量，检查是否嵌套，否则替换
    if classify_item(term1) == 0:
        if term1 in term2:
            return False, subst
        subst[term1] = term2
        return True, subst

    if classify_item(term2) == 0:
        if term2 in term1:
            return False, subst
        subst[term2] = term1
        return True, subst
    # 均非变量，则需完全一致才可合一
    return term1 == term2, subst
```

```
# 执行合一
def unify_literals(lit1, lit2):
    # 解析文字：判断是否为非，并获取谓词及参数列表
    def parse(lit):
        isneg = lit.startswith('~')
        body = lit[1:] if isneg else lit
        pred = body[:body.index('(')]
```

```

    args = body[body.index('(') + 1:-1].split(',')
    return isneg, pred, args

isneg1, pred1, args1 = parse(lit1)
isneg2, pred2, args2 = parse(lit2)
# 谓词不同或符号不同，不能归结
if pred1 != pred2 or isneg1 == isneg2:
    return False, {}
# 参数个数不同，不能归结
if len(args1) != len(args2):
    return False, {}
# 递归合一
subst = {}
for a1, a2 in zip(args1, args2):
    ok, subst = unify(a1, a2, subst)
    if not ok:
        return False, {}
return True, subst

```

```

# 将替换规则应用于整个子句
def apply_subst(clause, subst):
    res = []
    for lit in clause:
        isneg = lit.startswith('~')
        body = lit[1:] if isneg else lit
        pred = body[:body.index('(')]
        args = body[body.index('(') + 1:-1].split(',')
        new_args = [subst.get(a, a) for a in args]
        new_lit = f"{pred}({'.'.join(new_args)})"
        if isneg:
            new_lit = '~' + new_lit
        res.append(new_lit)
    return tuple(res)

```

(2) 归结：

```

# 遍历子句集，归结可归结子句并生成新子句
def resolve(clause1, clause2):
    clause1 = list(clause1)
    clause2 = list(clause2)

    for i, lit1 in enumerate(clause1):
        for j, lit2 in enumerate(clause2):
            ok, subst = unify_literals(lit1, lit2)
            if ok:
                new1 = [l for k, l in enumerate(clause1) if k != i]
                new2 = [l for k, l in enumerate(clause2) if k != j]
                new_clause = apply_subst(new1 + new2, subst)
                return True, new_clause, subst, (i, j)
    return False, (), {}, (-1, -1)

```

(3) 主归结算法：

```

# 输入知识库KB，判断是否证明成功并输出归结过程
def Resolution(KB):
    KB = list(KB)
    N = len(KB)
    resolution_step = []
    clause_set = set()
    max_len = max(len(c) for c in KB)
    # 初始化：将原始子句加入步骤
    for idx, clause in enumerate(KB):
        resolution_step.append({
            "id":idx,
            "clause":clause,
            "parents":[],
            "letters":[],
            "replacement":{}}
        })
        clause_set.add(clause)

    current_id = N
    proved = False
    # 循环归结直到推出空子句或证明失败
    while True:
        new_clauses = []
        for i in range(len(resolution_step)):
            for j in range(i + 1, len(resolution_step)):
                clause1 = resolution_step[i]["clause"]
                clause2 = resolution_step[j]["clause"]

                ok, res_clause, subst, (liti, litj) = resolve(clause1, clause2)
                if not ok:
                    continue

                # 得到空语句，证明成功
                if len(res_clause) == 0:
                    resolution_step.append({
                        "id":current_id,
                        "clause":res_clause,
                        "parents":[i,j],
                        "letters":([liti], [litj]),
                        "replacement":subst
                    })
                    proved = True
                    break

                # 新子句有效，加入集合
                if res_clause not in clause_set and len(res_clause) <= max_len:
                    clause_set.add(res_clause)
                    new_clauses.append((current_id, res_clause, [i,j], ([liti], [litj])))
                    current_id += 1

            if proved:
                break

        if proved:
            break

        if not new_clauses:
            break

    # 将新子句加入步骤
    for item in new_clauses:
        resolution_step.append({
            "id":item[0],
            "clause":item[1],
            "parents":item[2],

```

```

        "letters":item[3],
        "replacement":item[4]
    })
# 精简步骤+重新编号
result = backsearch(resolution_step,N)
result = renumber_ids(result,N)
# 生成输出
output = []
for idx,item in enumerate(result):
    clause_id = idx + 1
    clause = item["clause"]
    if not item["parents"]:
        line = f"{clause_id}({{'',''.join(clause)}})"
    else:
        p1, p2 = item["parents"]
        li, lj = item["letters"]
        subst = item["replacement"]
        rule = f"R[{p1 + 1}{chr(ord('a') + li[0])},{p2 + 1}{chr(ord('a') + lj[0])}]"
        if subst:
            s = ",".join([f"{k}={v}" for k, v in subst.items()])
            rule += f"{{{s}}}"
        line = f"{clause_id}{rule} = ({{'',''.join(clause)}})"
    output.append(line)
return output, proved

```

在代码实现中，使用了 `tuple` 存储子句以保证不可变性从而方便进行集合操作，使用 `set` 存储子句集以自动去重，防止搜索陷入死循环。

3. 创新点 & 优化

（1）证明路径的逆向追溯（Backsearch）：在归结函数 `resolve` 中可以看出，字句之间通过两两比较进行归结并生成替换规则集，这就说明过程中会产生 n^2 规模的新子句。代码通过 `backsearch` 函数，从最后产生的空子句出发，根据记录的 `parents` 属性逆向寻找支撑该结论的所有原始子句和中间步骤，并过滤掉搜索过程中产生的冗余无效子句，使输出的逻辑链条清晰简洁。

```

def backsearch(resolution_step, N):
    visited = set()
    st = []

    def trace(clause_id):
        if clause_id < N or clause_id in visited:
            return
        visited.add(clause_id)
        step = resolution_step[clause_id]
        st.append(step)
        for p in step["parents"]:
            trace(p)

    trace(len(resolution_step) - 1)
    return resolution_step[:N] + st[::-1]

```

(2) 自动重编号机制：在归结过程中，产生的子句 ID 往往是不连续的。代码通过 `renumber_ids` 函数，将最终选定的证明路径上的子句重新映射为连续的自然数，符合题目要求的展示格式。

```
def renumber_ids(result, N):
    all_ids = {it["id"] for it in result[N:]}
    id_map = {old: N + i for i, old in enumerate(sorted(all_ids))}
    id_map.update({i:i for i in range(N)})

    renumbered = []
    for item in result[N:]:
        new_item = {
            "id":id_map[item["id"]],
            "clause":item["clause"],
            "parents":[id_map[p] for p in item["parents"]],
            "letters":item["letters"],
            "replacement":item["replacement"]
        }
        renumbered.append(new_item)
    return result[:N] + sorted(renumbered, key=lambda x: x["id"])
```

(3) 启发式长度限制：为了避免子句长度过长导致性能退化，代码在归结时加入了 `len(res_clause) <= max_len` 的限制。这种启发式策略基于一个假设：如果证明存在，通常不需要生成比初始子句复杂得多的中间结论。

三、实验结果及分析

1. 实验结果展示

程序成功运行并解决了题目给出的两个逻辑推理问题：

```
1(A(tony))
2(~A(x),S(x),C(x))
3(~C(y),~L(y,rain))
4(~A(w),~C(w),S(w))
5(~L(tony,u),~L(mike,u))
6(L(tony,snow))
7(A(mike))
8(L(tony,v),L(mike,v))
9(L(tony,rain))
10(A(john))
11(L(z,snow),~S(z))
12R[2a,7a]{x=mike} = (S(mike),C(mike))
13R[4a,7a]{w=mike} = (~C(mike),S(mike))
14R[5a,6a]{u=snow} = (~L(mike,snow))
15R[11a,14a]{z=mike} = (~S(mike))
16R[12b,13a] = (S(mike),S(mike))
17R[15a,16a] = (S(mike))
18R[15a,17a] = ()
```

```
1(~On(xx,yy),~Green(xx),Green(yy))
2(Green(tony))
3(~Green(john))
4(On(mike,john))
5(On(tony,mike))
6R[1a,4a]{xx=mike,yy=john} = (~Green(mike),Green(john))
7R[1a,5a]{xx=tony,yy=mike} = (~Green(tony),Green(mike))
8R[2a,7a] = (Green(mike))
9R[3a,6b] = (~Green(mike))
10R[8a,9a] = ()
```

从输出结果可以看出，程序清晰地记录了每一步使用的归结规则R、参与归结的文字位置以及合一过程中使用的变量替换。

2. 评测指标展示及分析

测试用例	初始子句数	推导步数	运行时间（ms）	结论
Alpine Club	11	7	154.09ms	证明成功
On/Green Problem	5	5	25.83ms	证明成功

- 1. 计算效率： 通过import time模块测量运行时间，发现程序在处理中小规模的一阶逻辑问题时表现极佳，毫秒级即可完成证明。
- 2. 合一的正确性： 通过 Alpine Club 问题，验证了算法能够正确处理多层谓词嵌套及多个变量的同步替换。
- 3. 搜索策略： 采用的广度优先搜索保证了如果存在空子句，算法一定能找到它，但缺点是当知识库非常庞大时，产生的冗余子句会呈指数增长,例如Alpine Club的子句集中子句个数是On Problem的两倍，但是运行时间方面前者是后者的四倍以上。

四、 参考资料

课程课件：3-4-归结原理.pptx、归结原理讲解_v1.pptx